Git

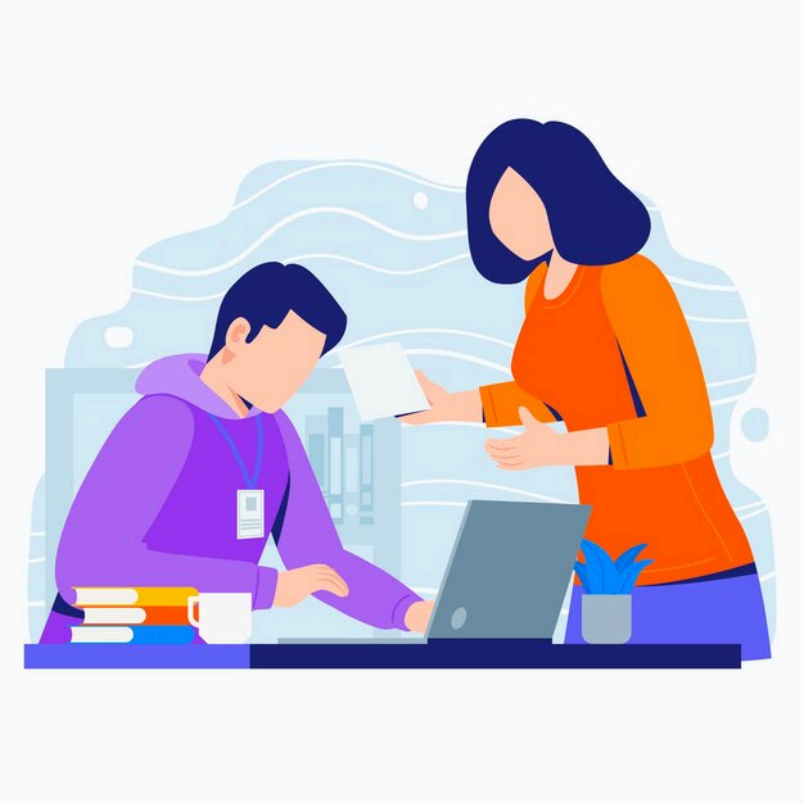
Лабораторная работа №1

Легенда:

Вы — стажер в компании "SimpleIT". В руках ароматный кофе и сегодня ваш первый рабочий день. Ваш тимлид выдает вам репозиторий с одним проектом по вычилению площадей фигур:

«Там площадь фигур. Прямоугольник криво считается, фигуры валятся на отрицательных сторонах, треугольника нет. Создай ветку, почини, добавь, покажи историю. Не сломай старое, ветку main не трогать!!! Удачи!»

Ваша задача — выполнить ряд действий над репозиторием создать новую ветку, добавить новые функции и исправить ошибки, не сломав старые, посмотрим историю и изучим внесенные изменения средствами Git



Дано:

Библиотека, написанная на Python:

https://github.com/E5capada/itmo_tools_2026.git

Подготовка:

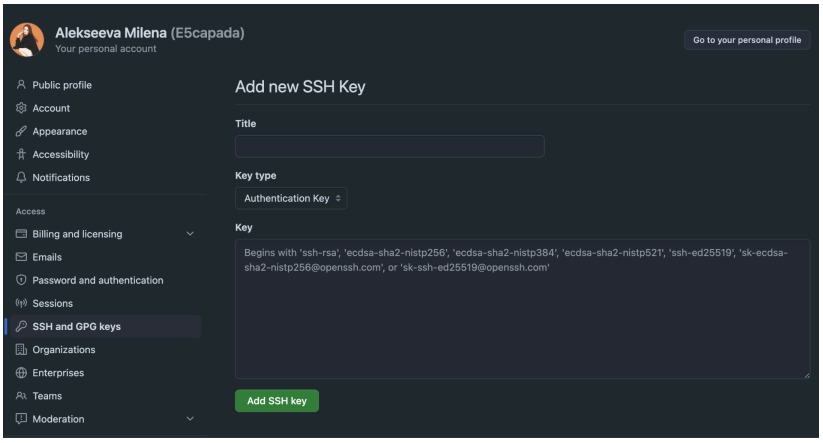
1. Зарегистрироваться на сайте: https://github.com/

Для авторицации по shh (если хотите подкючаться по ssh, а не по https):

1. (mac os) ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_github -C "your_email@example.com"

   (windows) Откройте PowerShell ssh-keygen -t rsa -b 4096

   ключи сохраняются в: C:\Users\Ваше_Имя_Пользователя\.ssh\
2. Добавить публичный **.pub** ключ https://github.com/settings/ssh/new :



3. В терминале ssh-add ~/.ssh/id_rsa_github добавить ключ для авторизации. Или ssh-add C:\Users\my_user_name\.ssh\id_rsa_github

Задание:

Цель:

Научиться базовым операциям Git для сохранения истории изменений, работы с ветками и совместной разработки, а также освоить навыки работы с терминалом

Уровень 1: Новичок (Базовый) — «Сохранение изменений в коде»

1. Клонирование репозитория
    - Выполните команду git clone:

      https://github.com/E5capada/itmo_tools_2026.git
2. Создание ветки
    - Создать новую ветку с названием **main_features_<ваш ID ISU>**
    - Переключитесь на нее.
3. Добавление первого файла
    - Например, с вычислениями для фигуры Прямоугольник.
    - Его название: rectangle.py
    - Его содержимое (например):

```
def area(a, b):
    return a * b


def perimeter(a, b):
    return a + b
```

    - Сделать коммит с сообщением о добавлении нового файла.
4. Добавление второго файла
    - Например, с вычислениями для фигуры Треугольник.
    - Его название: triangle.py
    - Его содержимое (например):

```python
def area(a, h):
    return a * h / 2

def perimeter(a, b, c):
    return a + b + c
```

5. Исправление ошибки
    - Найти и исправить ошибку в вычислении периметра в файле **rectangle.py**
6. Создание коммита с исправлением
    - Создайте коммит внутри этой же ветки, содержащий добавление файла и исправление (2 файла)
    - Сообщение коммита должно указывать на исправление ошибки
7. Анализ истории
    - Построить граф истории всего репозитория с однострочным выводом коммитов
    - Построить граф истории только текущей ветки (должно быть видно два последних коммита)
8. Просмотр изменений
    - Взять хэши двух последних коммитов из истории
    - Посмотреть, какие изменения были внесены в каждом из них

Уровень 2: Продвинутый (Средний) — «Слияние веток»

9. Переключение на основную ветку
    - Перейти в ветку main
10. Создание новой ветки
    - Создать новую ветку с названием **new_features_<ваш ID ISU>**
    - Переключиться на созданную ветку
11. Внесение изменений

- Добавить новую математическую функцию в файл **my_main_<ваш ID ISU>.py** (например, возведение в степень, вычисление факториала или другую операцию)
- Сделать коммит с изменениями

12. Слияние веток
    - Выполнить слияние ветки **new_features_<ваш ID ISU>** в **main_features_<ваш ID ISU>** ветку
13. Анализ истории
    - Построить граф истории всего репозитория с однострочным выводом коммитов
    - Построить граф истории только текущей ветки (должно быть видно два последних коммита)
14. Ломаем функцию
    - Сломать любую формулу в коде, например, перепишим формулу в square.py

```
def perimeter(a):
   return 3 * a // правильное 4*a
```

    - Сделать коммит с изменениями
15. Отменяем последний коммит
    - Используем **git revert**
16. Анализ истории
    - Построить граф истории только текущей ветки

Уровень 3: Профи (Сложный) — «Создаем конфликт»

17. Изменения в основной ветке
    - Перейти в ветку **main_features_<ваш ID ISU>** ветку

- В файле print.py присвойте переменной branch_name имя текущей ветки, пример:

```
def main():
    branch_name = 'main'
    print("this branch name:" + branch_name)
```

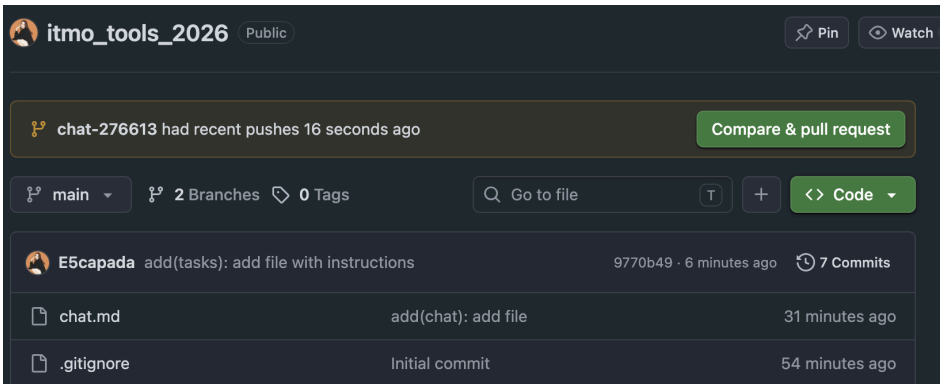
- Сделать коммит с изменениями

18. Изменения в дополнительной ветке
19. Перейти в ветку **new_features_<ваш ID ISU>** ветку
    - В файле print.py присвойте переменной branch_name имя текущей ветки, пример:

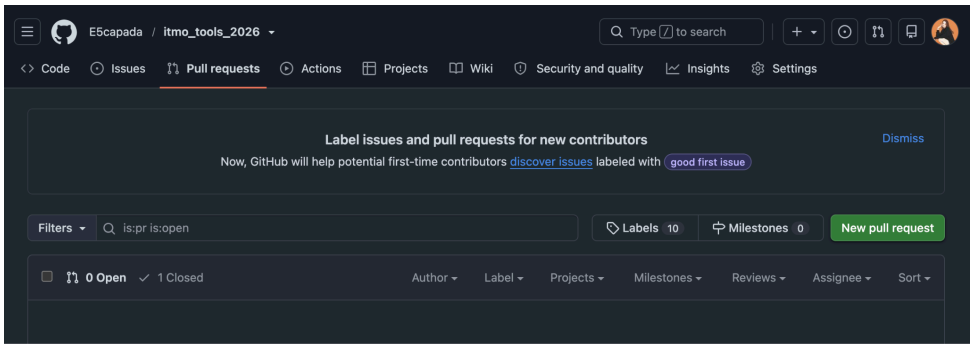
```
def main():
    branch_name = 'new'
    print("this branch name:" + branch_name)
```

    - Сделать коммит с изменениями
20. Слияние веток
    - Выполнить слияние ветки **new_features_<ваш ID ISU>** в **main_features_<ваш ID ISU>** ветку
21. Разрешить конфликт
22. Удалить временную ветку **new_features_<ваш ID ISU>**
23. Отправка изменения в основной проект
    - Перейти в ветку **main**
    - ⚠️Сделайте git pull
    - Создайте ветку git checkout -b **chat_<ваш ID ISU>**
    - В файле chat.md напишите свой комментарий
    - Сделайте коммит и опубликуйте ветку
    - git push
    - ⚠️Советую еще раз перед отправкой перейти в main, сделать git pull, перейти обратно в **chat_<ваш ID ISU>,** git merge main (для избежания конфликтов и сохранения актуальной версии)

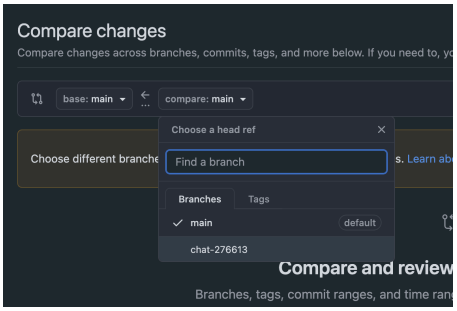
- Перейдите в репозиторий github создайте git pull request на merge **chat_<ваш ID ISU> в main**



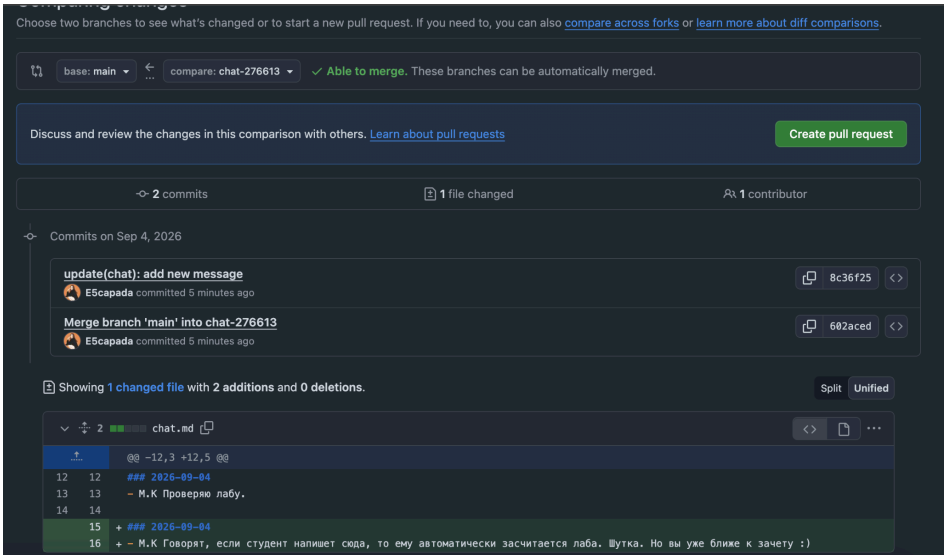
- Или переходим в pull requests, нажимаем new pull request

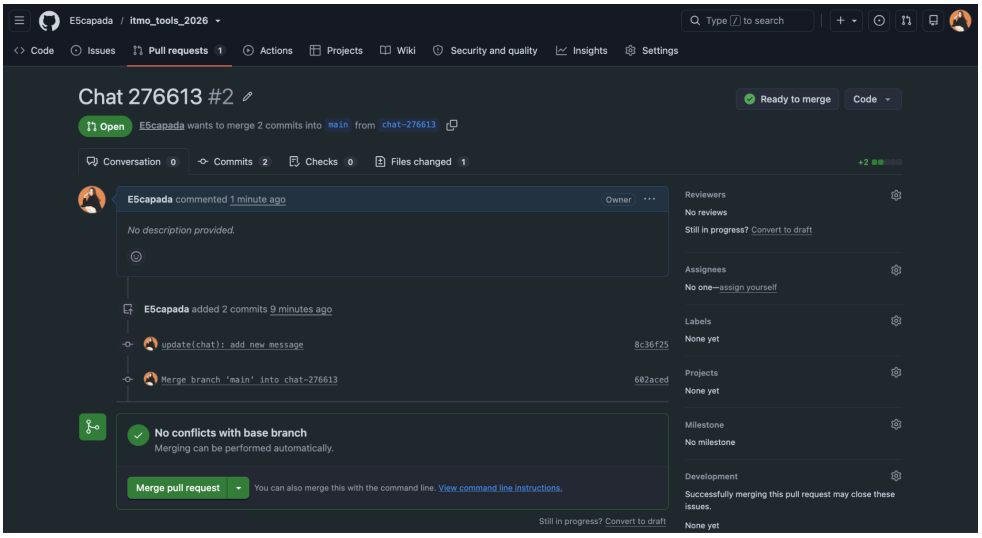


- Выбираем свою свою ветку main <- **chat_<ваш ID ISU>**



- Проверяем, что ничего не удалили чужого, нажимаем create pull request

- 

- Если все хорошо, разрешено сделать mege pull request
- 

- Теперь надо синхронизировать свой main с main в репозитории, чтобы получить все актуальные изменения
- git checkout main
- Смотрим в chat.md
- git pull
- Смотрим снова в chat.md

Требования для всех уровней:

Выполнение комманд через терминал/консоль

Требования к коммитам:

- Все коммиты должны иметь осмысленные сообщения на русском или английском языке
- Сообщения должны отражать суть внесенных изменений

Документирование:

- Сохранить все выполненные команды в отчет
- Приложить скриншоты внесенных изменений и графа истории
- Продемонстрировать возможность просмотра изменений между коммитами
- Описать возникшие проблемы и способы их решения

Оценивание

Уровень	Максимальный балл
Уровень 1 (Новичок)	10 баллов
Уровень 2 (Продвинутый)	3 балла
Уровень 3 (Профи)	2 балла
ВСЕГО	15 баллов

🎯 Примеры оценки

15/15 — выполнены все три уровня
10/15 — выполнен только Уровень 1 полностью
13/15 — выполнены Уровень 1 и Уровень 2

Полезные ссылки

- Репозиторий https://github.com/E5capada/itmo_tools_2026.git
- https://git-scm.com/ - git tool
- https://git-scm.com/book/ru/v2 или https://drive.google.com/file/d/1CM5ostn3uYdLcPvTgCdC9jLs9OkkgERb/view?usp=sharing - книга
- https://initialcommit.com/cluster/git-commands-git-cheat-sheets - команды
- https://dangitgit.com/ru#волшебная-машина-времени - что делать, если накосячил